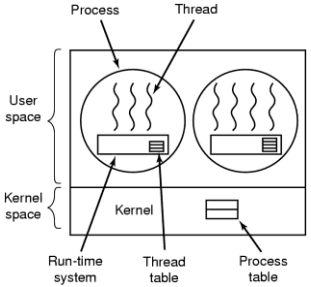
# SISTEMAS OPERATIVOS

## Aula Teórica 5

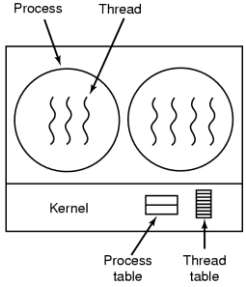
Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

# Implementing Threads




A user-level threads package




A threads package managed by the kernel

In user space each process needs its own private **thread table**, they are analogous to kernel's **process table**.

User-level threads also have other advantages, they allow each process to have its own customized scheduling algorithm. They also scale better, since kernel threads invariably require some table space and stack space in the kernel, which cam be a problem if there is a very latge number of threads.

Problems:

-Blocking system calls

-If a thread starts running, no other thread in the process will ever run unless the first thread voluntarily gives up the CPU.

-Programmers typically want threads precisely in applications where threads block often.

# Implementing Threads in the Kernel

There is no need for run-time system or thread table in each process.
The kernel has a thread table that keeps track of all threads in the system, holding all thread's registers, state, and other information. The information is the same as with user-level threads.

Kernel threads do not require any new, nonblocking system calls.
In addition if one thread in a process causes a page fault, the kernel can easily check to see if the process has any other runnable threads, and if so, run one of them while waiting for the required page to be brought from the disk.

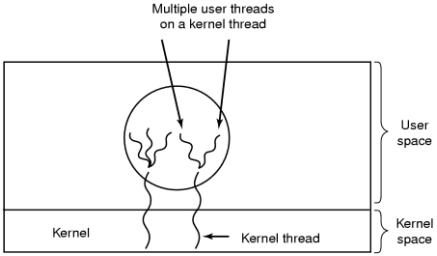
Their main disadvantage is that the cost of a system call is substancial, so if thread operation (creation, termination, etc) are common, much more overhead will be incurred.

Multithreaded process fork continous creating problems, and a lot of questions?

Another issue with threads is signals. In the classical model, signals are sent to processes only, not to threads.

In Linux, a signal may be handled by any thread, and the lucky winner is selected by the operating system.

# Hybrid Implementations




Multiplexing user-level threads onto kernel-level threads.

When this approach is used the programmer can determine
- how many kernel threads to use and
- how many user-level threads to multiplex on each one.

This model gives a ultimate flexibility.

The kernel is aware of only the kernel-level threads and thus schedules those. Some os those threads have multiple user-level threads.

User-level threads are created, destroyed, and scheduled just like user-level threads in a process that runs in a OS without multithreading capabilities.

# Making Single-Threaded Code Multithreaded (1)

Many existing programs were written for a single-threaded processes. Converting these to multithreading is much trickier than it may al first appear.

Local variables and parameters do not cause any trouble.

But variables that are global to a thread but not global to the entire program **are a problem**.

Example: When a process or thread makes a system call taht fails, the error code is out into *errno* variable.
– T1 execution of a system call Acess to find out if it has permission to acess a certain file, the operating system return the answer in the global variable *errno*.
But before T1 had the chance to read *errno*, the schedular decides to switche to T2.
-T2 executes a open call that fails, whichs causes *errno* to be overwritten, and thread T1 access code will be lost forever.
- When T1 starts up later, it will read the wrong value and behave incorrectly.

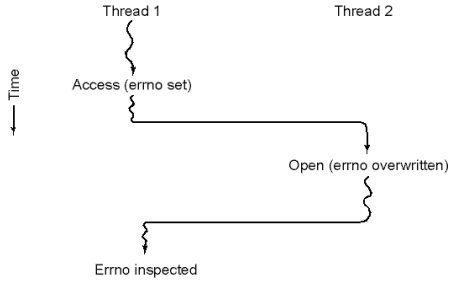



Fig. 2-17. Conflicts between threads over the use of a global variable.

# Making Single-Threaded Code Multithreaded (2)

## Possible solutions

-One is to prohibit global variables altogether.

-Assign each thread its own private global variables.
ex: own private copy of *errno*, and other global variables.

New library procedures can be introduced to create, set and read these threadwide global variables.

```
create_global("bufptr");
set_global("bufptr",&buf);
bufptr=read_global("bufptr");
```

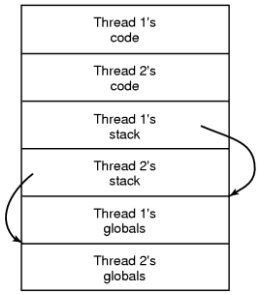



Fig. 2-18. Threads can have private global variable.

## Other Problems

- Many library procedures are not reentrant.
- Some signals are logically thread specific, others are not.
- Stack management

# Event-Driven Servers(1)

In the example of the web server, we seen two possible designs:

- fast multithreaded;
- slow single-threaded.

If nonblocking versions of system calls, such as read, are available, a third approach is possible - **finite-state machine**.

When a request comes in, the one and only thread examines it.
If it can be satisfied from cache, fine, if not, a **nonblocking disk operation starts**.
The server records the state of the current request in a table and then goes and get the next event.
The next event may either be a request for new work or a replay from the disk abaout previous operation.
If it is new work, that work started.
If it is a reply from de disk, the relevant information is fetched from the table and replay processed. With nonblocking disk I/O, a reply probably will have to take the form of a signal or interrup.

In a design like this the "sequential process" model is lost. Every computation has a saved state, and there exists some set of events that can occur to change the state – **even-driven programming**.

Every event leads to a burst of activity, **but it never blocks.**

A single-threaded event-driven server can handle many clientes concurrently.

```
0.  /* Preliminaries:
1.     svrSock   : the main server socket, bound to TCP port 12345
2.     toSend    : database to track what data we still have to send to the client
3.                 - toSend.put (fd, msg) will register that we need to send msg on fd
4.                 - toSend.get (fd) returns the string we need to send msg on fd
5.                 - toSend.destroy (fd) removes all information about fd from toSend */
6.
7.  inFds      = { svrSock }            /* file descriptors to watch for incoming data */
8.  outFds     = { }          /* file descriptors to watch to see if sending is possible */
9.  exceptFds  = { }          /* file descriptors to watch for exception conditions (not used) */
10.
11. char msgBuf [MAX_MSG_SIZE]      /* buffer in which to receive messages */
12. char *thankYouMsg = "Thank you!"  /* reply to send back */
13.
14. while (TRUE)
15. {
16.       /* block until some file descriptors are ready to be used */
17.       rdyIns, rdyOuts, rdyExcepts = select (inFds, outFds, exceptFds, NO_TIMEOUT)
18.
19.       for (fd in rdyIns) /* iterate over all the connections that have something for us */
20.       {
21.            if (fd == svrSock)                    /* a new connection from a client */
22.            {
23.                 newSock = accept (svrSock)      /* create new socket for client */
24.                 inFds = inFds ⋃ { newSock }     /* must monitor it also */
25.            }
26.            else
27.            {    /* receive the message from the client */
28.                 n = receive (fd, msgBuf, MAX_MSG_SIZE)
29.                 printf ("Received: %s:0, msgBuf)
30.
31.                 toSend.put (fd, thankYouMsg)    /* must still send thankYouMsg on fd */
32.                 outFds = outFds ⋃ { fd }        /* so must monitor this fd */
33.            }
34.       }
35.       for (fd in rdyOuts) /* iterate over all the connections that we can now thank */
36.       {
37.            msg = toSend.get (fd)     /* see what we need to send on this connection */
38.            n = send (fd, msg, strlen(msg))
39.            if (n < strlen (thankYouMsg)
40.            {
41.                 toSend.put (fd, msg+n)  /* remaining characters to send next time*/
42.            } else
43.            {
44.                 toSend.destroy (fd)
45.                 outFds = outFds \ { fd }  /* we have thanked this one already */
46.            }
47.       }
47. }
```

**Figure 2-19.** An event-driven thank-you server (pseudo code).

A main server socket (e.g., ServerSocket in Java or a bound TCP socket in C/Python) is a network endpoint that listens for incoming client connection requests on a specific port. It acts as the "master" socket that accepts connections and typically hands them off to child sockets for data exchange, allowing the main socket to continue listening for new requests.

## Compare the three diferente ways to build a server

Threads make it possible to retain the idea of sequential processes that make blocking calls (e.g. for disk I/O) and still achieve parallelism. Blocking system calls make programming easier, and parallelism improves performance.

A single–threaded server retains the simplicity of blocking system calls but gives up performance.

Event-driven programming, also achieves high performance through parallelism but uses nonblocking calls and interrups to do so. It is considered harder to program.

The Linux Kernel in modern Intel CPUs is a multi-threaded operating system kernel.
In contrast, MINIX 3, consists of many servers implemented following the model of finite state machine and events.

| Model | Characteristics |
|---|---|
| Threads | Parallelism, blocking system calls |
| Single-threaded process | No parallelism, blocking system calls |
| Finite-state machine/event-driven | Parallelism, nonblocking system calls, interrupts |

**Figure 2-20.** Three ways to construct a server.

# QUESTÕES?

Daniel Castro
dcastro@ualg.pt

